PrepPilot: An AI-Powered Adaptive Learning and Interview Preparation System for Students

[Author Name]
[Department], [Institution], [City, Country] | [Email Address]

Abstract

Students in higher education must prepare for academic assessments while also developing the technical knowledge and communication skills required for recruitment interviews. Managing these activities independently can make it difficult to identify knowledge gaps, prioritize topics, monitor progress, and allocate study time. This paper presents PrepPilot, a student-focused system that combines academic preparation and interview practice in a unified environment. The documented prototype provides subject and topic management, quiz activities, revision recommendations, interview question practice, mock-interview workflows, progress analytics, readiness indicators, and an assistant interface. Its topic-prioritization mechanism combines topic status, prior quiz performance, importance, interview relevance, learner confidence, and time remaining before relevant deadlines. Separate academic and interview readiness indicators summarize different dimensions of preparation. This paper describes the problem context, related research, requirements, prioritization method, system modules, implementation characteristics, and a feature-level evaluation approach. The available implementation evidence supports the presence of the core workflows; however, no controlled user study or quantitative outcome dataset is available for this paper. Therefore, the paper does not claim that PrepPilot improves examination marks, placement rates, or interview success. The system is presented as a practical preparation-support tool whose usability and educational effectiveness should be evaluated in future work.

Index Terms—Adaptive learning, personalized learning, educational technology, interview preparation, topic prioritization, readiness assessment, student support system.

I. INTRODUCTION

A. Background

Students in higher education are expected to demonstrate academic proficiency while also preparing for internships, placements, and employment opportunities. Academic preparation includes organizing course subjects, understanding topics, revising difficult concepts, completing assessments, and monitoring progress toward examinations. Interview preparation adds further tasks, including technical question practice, communication development, behavioral questions, and reflection on previous answers. Although these activities are related, students may manage them through separate notes, learning platforms, question banks, and personal schedules.

Digital learning systems can help organize these activities and provide feedback based on learner information. Personalized adaptive learning seeks to tailor learning pathways or support to individual needs. A scoping review by du Plooy, Casteleijn, and Franzsen examined the characteristics of personalized adaptive learning in higher education and summarized reported relationships with academic performance and engagement [1]. This literature motivates the investigation of systems that use learner-related information to guide preparation. It does not, however, establish that every adaptive feature will produce better outcomes in every context.

The design and evaluation of adaptive learning systems also require attention to stakeholder needs. Alajlani, Crabb, and Murray reviewed 35 studies concerning stakeholder involvement in adaptive learning system development and identified areas where educator participation could be strengthened [2]. More broadly, Bond et al. called for increased ethics, collaboration, and methodological rigor in research on artificial intelligence in higher education [3]. These findings support a cautious approach: a preparation platform should make its recommendations understandable and should be evaluated on the basis of observable functionality and suitable evidence.

B. Problem Statement

Students preparing for examinations and recruitment interviews face several connected challenges. First, they must organize a potentially large set of subjects and topics. Second, they need to distinguish between topics that are well understood and topics that require additional revision. Third, priorities can change when new assessment results are recorded or when an examination or interview approaches. Finally, academic and interview preparation require different forms of practice, so a single undifferentiated progress measure may not explain where further work is needed.

When these activities are tracked separately, students may lack a consistent overview of their progress and may have difficulty deciding what to study next. The problem addressed in this paper is the design and implementation of a unified system that records preparation information, prioritizes topics, identifies revision needs, supports interview practice, and presents interpretable indicators of preparation status. PrepPilot is intended to assist student planning; it is not intended to replace teachers, professional career guidance, or independent judgment.

C. Proposed System

PrepPilot (Preparation Pilot) is a student-focused application that combines academic preparation and interview practice. Its main areas include a dashboard, subject and topic management, quizzes, revision, interview preparation, mock interviews, analytics, and an assistant interface. The dashboard presents topic progress, recommended priorities, recent quiz activity, and academic and interview readiness indicators.

The central prioritization mechanism assigns each topic a score based on multiple factors. The documented prototype considers whether a topic is incomplete, the most recent quiz score, topic importance, interview relevance, deadline urgency, and self-reported confidence. Topics are then ordered by score and grouped into high, medium, and low priority levels. The revision view identifies topics using conditions such as an explicit revision status, a low quiz score, low confidence, or a long interval since studying an important completed topic.

PrepPilot also distinguishes academic readiness from interview readiness. Academic readiness draws on topic completion, quiz performance, weak-topic counts, and confidence ratings. Interview readiness draws on practice-question volume, heuristic answer scores, completed mock interviews, and mock-interview scores. The overall indicator combines the two readiness values using the weighting implemented in the prototype. These scores are application-defined summaries, not validated predictions of grades or employment outcomes.

D. Objectives

1. To provide a centralized interface for organizing subjects, topics, quizzes, revision, and interview practice.
2. To prioritize topics using learner progress, performance, importance, confidence, interview relevance, and deadline information.
3. To identify topics that may require revision and explain the recorded reasons for each recommendation.
4. To summarize academic and interview preparation separately and provide a consolidated progress indicator.
5. To provide analytics and assistant-oriented support for interpreting preparation information.
6. To document the implemented workflows and identify appropriate functional and empirical evaluation methods.

E. Contributions and Scope

The first contribution is the integration of academic topic tracking, assessment, revision support, and interview practice within one student-oriented application. The second is a transparent, multi-factor topic-prioritization method that combines learner-related signals with time-based urgency. The third is the separation of academic and interview readiness indicators while retaining a shared dashboard. These contributions concern system design and implementation; they should not be interpreted as evidence of a new learning theory or a validated artificial-intelligence model.

The scope of this paper is feature-level system design and implementation. The available browser prototype is implemented with HTML, CSS, and JavaScript and stores its state in browser local storage. Its quiz generation uses templates, and its assistant responses are rule-based rather than generated by a live large language model. Some broader full-stack development work has also been explored, but the exact production architecture and deployment configuration are not asserted here unless confirmed from the final source code. This distinction prevents prototype behavior from being incorrectly described as a deployed AI service.

F. Organization of the Paper

Section II reviews relevant research. Section III presents requirements and the proposed methodology. Section IV describes the system architecture and module interactions. Section V documents implementation characteristics. Section VI outlines feature-level testing and discusses the limits of the available evaluation evidence. Section VII concludes the paper and identifies future work.

II. RELATED WORK

A. Personalized Adaptive Learning

Personalized adaptive learning attempts to adjust learning experiences to learner needs rather than presenting every learner with an identical path. In their scoping review, du Plooy et al. examined the characteristics and reported impacts of personalized adaptive learning in higher education [1]. Their synthesis indicates potential benefits but also discusses practical and technological limitations. For PrepPilot, the relevant design implication is that learner-related data can be used to inform support decisions, while claims about learning impact must remain tied to measured outcomes.

PrepPilot applies a narrower form of adaptation than a system that dynamically changes instructional content or models a learner's knowledge in depth. It ranks topics using explicit input fields and presents revision reasons. This approach is comparatively interpretable: a student can inspect whether a recommendation is driven by an incomplete status, a low quiz score, importance, confidence, or an approaching deadline. Its limitation is that the quality of recommendations depends on the accuracy of the recorded data and the suitability of the chosen weights.

B. Development and Evaluation of Adaptive Systems

Alajlani et al. reviewed stakeholder participation in the development of adaptive learning systems and identified a need for stronger educator involvement across design, implementation, and evaluation [2]. This is relevant to student-facing tools because topic importance, revision thresholds, and readiness indicators should reflect real study practices rather than being treated as universally valid constants. PrepPilot currently encodes these decisions as software rules. Future evaluation should therefore include students and, where feasible, educators or career advisers to determine whether the recommendations are understandable and useful.

C. Artificial Intelligence in Higher Education

Bond et al. conducted a meta-systematic review of artificial intelligence in higher education and emphasized ethics, collaboration, and rigor in the research base [3]. For PrepPilot, this supports a clear distinction between an interface described as AI-powered and the capabilities that are actually active in a given build. The documented prototype's assistant uses predefined rule-based responses, and its quiz questions are generated through templates. These mechanisms can demonstrate workflow and interaction design, but they should not be presented as evidence of live generative AI, model training, or validated automated assessment.

D. Research Gap and Positioning

The cited literature establishes the relevance of personalization and the importance of careful design and evaluation, but it does not establish the effectiveness of PrepPilot itself. The system's practical focus is the combination of academic topic prioritization and interview preparation in a single student workflow. The paper's contribution is therefore positioned as an implemented system design with explicit prioritization logic and separate preparation indicators. A comparative study against existing tools or a controlled learner study would be required to establish whether this integration provides measurable advantages.

III. REQUIREMENTS AND METHODOLOGY

A. Intended Users and Requirements

The intended users are students who need to track academic topics and prepare for interviews. The primary functional requirements are: (1) create and manage subjects and topics; (2) record topic status, importance, interview relevance, confidence, and dates; (3) create and submit quizzes; (4) identify revision topics; (5) practice technical, project-related, human-resources, and behavioral interview questions; (6) record practice and mock-interview feedback; and (7) view progress and readiness indicators.

Non-functional considerations include usability, responsiveness, consistency of stored state, explainability of recommendations, and appropriate handling of student data. Since the documented prototype stores data locally in the browser, its data persistence is tied to that browser profile and device. This behavior is suitable for a demonstrator but does not provide multi-device synchronization or centralized account-based storage by itself.

B. Topic-Prioritization Method

The documented prototype computes a score between 0 and 100 for each topic. Six weighted components contribute to the score: incompleteness (25 points), weakness based on the most recent quiz score (25), topic importance (15),

interview relevance (15), deadline urgency (15), and confidence gap (5). The weights sum to 100. Inputs are normalized to the component ranges, and the final score is rounded and bounded between 0 and 100.

Let I represent the incomplete-status fraction, Q the weakness fraction, M the normalized importance, R the normalized interview relevance, U the urgency fraction, and C the confidence-gap fraction. The score is defined as:

$$\mathbf{P = round(25I + 25Q + 15M + 15R + 15U + 5C)}$$

The incompleteness component depends on topic status: not started receives the full fraction, in progress receives half, a topic marked for revision receives 0.85, and completed receives zero. The weakness component is based on (100 − last quiz score)/100; it contributes zero when no quiz score exists. Importance and interview relevance are normalized from a 1–5 scale. Deadline urgency uses piecewise values that increase as an examination or relevant interview approaches. Interview-date urgency is considered for topics whose interview relevance is at least 3. The confidence-gap component increases as the learner's self-rating decreases; when confidence has not been rated, the prototype uses a neutral midpoint.

The resulting score is categorized as HIGH when it is at least 65, MEDIUM when it is at least 35 but below 65, and LOW otherwise. The system also records human-readable reasons, such as a topic being incomplete, a score below 70%, low confidence, high importance, or an approaching deadline. The reasons are intended to make the recommendation easier to understand. The score is a heuristic ranking function; its weights and thresholds have not been calibrated through a controlled learner study.

C. Revision Identification

The revision list is computed from topic records. A topic is flagged if it is explicitly marked as needing revision, has a recorded quiz score below 70%, has a confidence rating of 2/5 or lower, or is an important completed topic that has not been studied for at least 21 days. Multiple reasons may be shown for a single topic. The flagged topics are then ordered using the topic-priority calculation. These thresholds are implementation rules rather than universal educational standards and may require adjustment following user feedback.

D. Readiness Indicators

Academic readiness combines four factors: topic completion (40% of the academic score), quiz performance (30%), the proportion of topics not classified as weak (15%), and average self-reported confidence (15%). In the documented implementation, a topic is considered weak when its last recorded quiz score is below 70% or its confidence rating is 2/5 or lower. If no topics exist, academic readiness is set to zero.

Interview readiness combines question-practice volume (20%), average heuristic practice-answer score (25%), completed mock-interview volume (15%), and average mock-interview score (40%). Practice volume saturates at 20 answered questions and mock-interview volume at three completed sessions. The overall preparation indicator is calculated as 55% academic readiness and 45% interview readiness. These exact weights describe the prototype's current scoring logic; they do not represent validated predictors of student outcomes.

E. Quiz and Interview Feedback

The documented prototype generates quiz items from templates associated with a selected subject and topic. It supports multiple-choice, true/false, and short-answer item formats. The short-answer checker uses keyword matching rather than semantic understanding. Consequently, a response may receive a positive or negative result that does not fully reflect its meaning. Interview practice uses a predefined question bank and evaluates answer characteristics such as keyword coverage and response length. The resulting scores are best understood as formative, rule-based feedback rather than expert grading.

These design choices allow the application to demonstrate the end-to-end workflow of practice, feedback, and progress tracking without requiring a live external AI service. A future version could integrate a language model behind a secure server-side API, but that integration must be documented as implemented only after it is connected, tested, and included in the evaluated build.

IV. SYSTEM DESIGN AND ARCHITECTURE

A. Logical Architecture

At the logical level, PrepPilot can be divided into four layers: (1) the user-interface layer, (2) the application-logic layer, (3) the preparation-data layer, and (4) the feedback and analytics layer. The user interface contains the

dashboard and feature views. Application logic manages topic prioritization, revision detection, quiz workflows, interview feedback, and readiness calculations. The data layer stores subjects, topics, assessment records, practice logs, preferences, and history. The feedback layer converts those records into priority lists, explanations, charts, and readiness indicators.

In the documented browser prototype, these responsibilities are implemented within a client-side HTML/CSS/JavaScript application, and the state is serialized into browser local storage. The logical layers describe responsibilities, not independently deployed services. They should not be interpreted as evidence that the prototype uses a separate production database, a remote AI service, or a cloud deployment.

Presentation layer Dashboard • Subjects and Topics • Quizzes • Revision • Interview Prep • Analytics • Assistant
Application logic Topic priority • Revision rules • Quiz scoring • Interview feedback • Readiness calculation
State and data Profile and dates • Subject/topic records • Quiz history • Practice log • Mock interviews • Preferences
Outputs Priority ranking • Revision reasons • Progress analytics • Academic/interview readiness

Fig. 1. Logical view of PrepPilot's main responsibilities. The diagram is conceptual and does not imply separate deployed services.

B. Main Modules

Dashboard. Summarizes preparation status, shows readiness indicators, highlights prioritized topics, and provides navigation to key workflows.

Subject and topic management. Allows the student to create subjects and topics and record status, importance, interview relevance, confidence, and related dates.

Quiz module. Creates a quiz from a selected topic, subject, weak-topic set, or mixed review; records answers and calculates a result using the prototype's rule-based checks.

Revision module. Flags topics based on status, quiz performance, confidence, and elapsed time since studying selected important topics.

Interview preparation. Organizes predefined technical, project, HR, and behavioral questions and records practice answers and feedback.

Mock interviews. Provides a workflow for interview practice sessions and stores completion and score information used by the readiness calculation.

Analytics. Presents recorded progress and performance information to help the student inspect preparation history.

Assistant interface. Responds to a limited set of preparation-related intents through deterministic rules in the documented prototype.

C. Data Flow

A typical workflow begins when a student adds subjects and topics and enters attributes such as importance, confidence, status, and relevant dates. The application evaluates these records to calculate topic priorities and identify revision needs. A student can then choose a quiz or interview practice activity. The resulting score or feedback is stored in the application's state and can influence later revision and readiness calculations. The dashboard presents a summary of the updated state. This creates a feedback loop between activity, recorded evidence, prioritization, and subsequent study decisions.

D. Design Considerations

The use of explicit rules improves interpretability because the system can display reasons for a priority score. The trade-off is that heuristic rules cannot fully model knowledge, forgetting, topic dependencies, or differences between assessment types. Readiness scores may also be sensitive to the quantity and quality of input data. A student with little recorded practice may receive a low indicator partly because the system lacks evidence, not necessarily because the student lacks ability. The interface should communicate this distinction and avoid presenting the scores as objective measures of competence.

V. IMPLEMENTATION

A. Prototype Technologies and State Management

The source artifact documented for this paper is a browser-based application written with HTML, CSS, and JavaScript. It renders feature views in the browser and uses JavaScript functions to manage navigation, forms, calculations, and user interactions. The current state is saved as a serialized object in local storage and restored when the application is opened again in the same browser profile. A reset operation restores the seeded demonstration state. This implementation allows local demonstration without requiring a remote database for the prototype.

The state model includes profile settings, subjects, topics, quizzes, mock interviews, practice logs, notifications, readiness history, and records of previously flagged topics. Subject records contain topic collections, while topic records include status, difficulty, importance, interview relevance, estimated study time, confidence, last quiz score, and last-studied date. Quiz and interview records support the feedback and analytics views.

B. Topic Prioritization and Revision

The prioritization function maps each topic to a score and explanatory reasons, then sorts the collection in descending score order. It also estimates a study duration from the topic's estimated minutes, priority level, and difficulty. In the prototype, HIGH priority uses the full estimated duration, MEDIUM uses 75%, and LOW uses 50%, with a minimum recommendation of 15 minutes. Difficulty adjusts the duration using a multiplier: 1.15 for advanced topics, 0.90 for beginner topics, and 1.00 for intermediate topics. These estimates are planning aids and are not based on measured individual learning speed.

Revision recommendations are recalculated from current topic data. The application stores notification records when a topic becomes newly flagged, and it removes a topic from the set of seen weak topics when the topic no longer satisfies a revision condition. This makes the revision view responsive to changes in topic status, confidence, or quiz score.

C. Assessment and Interview Workflows

Quiz generation in the documented prototype is template-based. Question content is constructed from the selected subject and topic names, with a mixture of multiple-choice, true/false, and short-answer formats. This supports interface testing and workflow demonstrations, but template questions are not a substitute for a curated question bank or validated assessment instrument. In particular, generated distractors and explanations should be reviewed before the quiz feature is used for high-stakes learning decisions.

Interview questions are drawn from a predefined bank grouped into technical, project, HR, and behavioral categories. The answer-evaluation routine uses keyword coverage and answer length to derive several feedback dimensions, including correctness, relevance, completeness, clarity, depth, and communication. Since these dimensions are estimated by rules, the output can be sensitive to wording and may not reliably judge semantic correctness. The feature is therefore presented as practice feedback, not as a replacement for human assessment.

D. Assistant and AI Scope

Although the project title uses the term 'AI-Powered', the documented prototype assistant is rule-based. It recognizes a limited set of question patterns, such as asking what to study, what to revise, or how ready the student is, and constructs a response from the current application state. It does not demonstrate that a live large language model is connected or that a model has been trained for this project. Any separate full-stack implementation or future model integration should be described only after its active behavior and dependencies are verified in the final build.

E. Privacy and Deployment Boundaries

The browser prototype stores its data locally, which avoids sending the prototype's state to a server as part of its local-storage mechanism. However, local storage is not encrypted storage and can be accessed by scripts running in the same origin. It is also tied to the browser and device. A production deployment would need a defined authentication and authorization model, secure server-side data storage, input validation, backup and deletion policies, transport security, and appropriate protection for any API credentials. If a third-party AI service is introduced, the system should avoid sending unnecessary personal data and should clearly disclose the processing involved.

VI. TESTING AND EVALUATION

A. Evaluation Approach

The available project description indicates that the core features have been implemented and tested during development. The evidence available for this manuscript does not include a formal test report, a measured user study, or a benchmark dataset. Accordingly, this section describes the appropriate feature-level evaluation scope and reports no invented pass rate, timing result, user satisfaction score, or learning gain. The final submission should be updated with the actual test log if one is available.

Functional evaluation should verify that each main workflow produces the expected state change and user-visible output. Algorithmic evaluation should check whether changes in input factors affect topic priorities in the intended direction. Data-persistence evaluation should confirm that state is retained across refreshes in the same browser and that reset behavior works as intended. Usability evaluation should examine whether students can find features, understand recommendation reasons, and distinguish readiness estimates from validated performance measures.

B. Feature-Level Test Scenarios

ID	Test scenario	Expected behavior	Evidence/status
T1	Create, edit, and delete a subject or topic	The record is added, updated, or removed and the displayed subject/topic list is refreshed.	Feature-level test; insert observed result from test log.
T2	Change topic status	The priority score changes according to the configured status fraction.	Deterministic rule; verify in final build.
T3	Record a low quiz score	The topic receives a weakness contribution and can appear in the revision list.	Deterministic rule; verify in final build.
T4	Lower confidence rating	The confidence-gap contribution increases and a low-confidence revision reason can appear.	Deterministic rule; verify in final build.
T5	Move a deadline closer	The urgency contribution increases according to the configured urgency bands.	Deterministic rule; verify in final build.
T6	Submit a quiz	Answers are checked, a result is stored, and the quiz history is updated.	Feature-level test; no aggregate pass rate available.
T7	Submit an interview practice answer	The heuristic feedback and score are displayed and stored in practice history.	Feature-level test; scoring validity not established.
T8	Refresh the browser	The local state is restored from local storage in the same browser profile.	Persistence behavior supported by prototype implementation.
T9	Reset demonstration data	The seeded demonstration state is restored.	Reset workflow supported by prototype implementation.
T10	Inspect readiness values	Academic, interview, and overall indicators are calculated from recorded state using documented weights.	Calculation can be unit-tested; no independent benchmark supplied.

Table I. Proposed verification matrix for the documented prototype. Expected behavior is based on the implemented rules; entries marked for verification should be replaced with observed outcomes from the final build's test log.

C. Interpretation of Evaluation Evidence

The implementation supports a coherent workflow from topic entry to priority ranking, revision, assessment, interview practice, and readiness summaries. The deterministic nature of the priority and readiness calculations makes them suitable for unit testing with controlled inputs. Nevertheless, correct execution of a formula does not demonstrate that its weights are educationally optimal. Likewise, a working quiz or interview interface does not establish the validity of its generated questions or scoring method. Those questions require expert review, representative test cases, and user-centered evaluation.

A subsequent evaluation could recruit student volunteers to complete a set of preparation tasks and assess task completion, error frequency, perceived usability, and the clarity of recommendations. A separate study would be needed to compare learning outcomes, for example by measuring pre-test and post-test performance across an appropriate comparison design. Any such study should define participants, instruments, consent procedures, and analysis methods before collecting data.

D. Limitations

- The prioritization weights and thresholds are hand-defined and have not been empirically calibrated.
- The prototype's quiz generator is template-based, and short-answer checking relies on simple keyword matching.
- The interview-answer scoring method is heuristic and should not be interpreted as expert or validated AI assessment.
- The assistant in the documented prototype is rule-based rather than a connected generative AI model.

- Local browser storage does not provide account synchronization, centralized backup, or production-grade data protection.
- No controlled user study or quantitative learning-outcome dataset is available for this manuscript.

VII. CONCLUSION AND FUTURE WORK

This paper presented PrepPilot, a student-focused system that combines academic preparation and interview practice within a single application. The documented implementation includes subject and topic management, multi-factor topic prioritization, revision recommendations, quizzes, interview practice, mock-interview workflows, analytics, readiness indicators, and a rule-based assistant. The prioritization method considers topic status, previous quiz performance, importance, interview relevance, deadline urgency, and confidence. Separate readiness calculations provide a consolidated view of academic and interview preparation.

The main contribution is an integrated, interpretable preparation workflow rather than a claim of proven educational impact or a newly trained AI model. The available implementation demonstrates how explicit rules can organize learner-recorded information into priorities and feedback. At the same time, the prototype's template-based quizzes, heuristic scoring, manually selected weights, and local storage impose important limitations. No claim is made that the system improves academic marks, interview performance, or placement outcomes.

Future work should begin with systematic functional and unit testing, followed by student and educator feedback on the usefulness of priority explanations and readiness indicators. The quiz bank and answer evaluation should be improved and validated. The prioritization weights could be tuned using appropriately collected data, while a live AI integration could be added through a secure backend if required. A production version should also provide robust authentication, secure persistence, privacy controls, and clear disclosure of how student data is processed. Finally, a controlled user study would be necessary to determine whether PrepPilot improves study planning, engagement, or learning outcomes compared with existing preparation methods.

REFERENCES

[1] E. du Plooy, D. Casteleijn, and D. Franzsen, “Personalized adaptive learning in higher education: A scoping review of key characteristics and impact on academic performance and engagement,” Heliyon, vol. 10, no. 21, Art. no. e39630, 2024, doi: 10.1016/j.heliyon.2024.e39630.

[2] N. Alajlani, M. Crabb, and I. Murray, “A systematic review in understanding stakeholders’ role in developing adaptive learning systems,” Journal of Computers in Education, vol. 11, pp. 901–920, 2024, doi: 10.1007/s40692-023-00283-x.

[3] M. Bond, H. Khosravi, M. De Laat, N. Bergdahl, V. Negrea, E. Oxley, P. Pham, S. W. Chong, and G. Siemens, “A meta systematic review of artificial intelligence in higher education: A call for increased ethics, collaboration, and rigour,” International Journal of Educational Technology in Higher Education, vol. 21, Art. no. 4, 2024, doi: 10.1186/s41239-023-00436-z.